

Core Java Interview Questions with Answers and Examples

1. OOPs Concepts

What are the 4 pillars of OOP?

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

Encapsulation

Binding data and methods together.

```
class Employee {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Abstraction

Showing only essential details.

```
abstract class Animal {  
    abstract void sound();  
}
```

Inheritance

One class acquiring properties of another.

```

class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
}

```

Polymorphism

One method behaving differently.

```

class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
    void show() {
        System.out.println("B");
    }
}

```

2. Abstraction vs Encapsulation

Abstraction	Encapsulation
Hides implementation	Hides data
Achieved using abstract class/interface	Achieved using private variables
Focuses on behavior	Focuses on security

3. Interface vs Abstract Class

Interface	Abstract Class
Supports multiple inheritance	Does not support multiple inheritance

Interface	Abstract Class
Variables are public static final	Can have instance variables
Methods are abstract by default	Can have abstract and concrete methods

Example:

```
interface Vehicle {
    void start();
}

abstract class Car {
    abstract void drive();
}
```

4. Method Overloading vs Overriding

Overloading

Same method name with different parameters.

```
class MathUtil {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Overriding

Child class changes parent behavior.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
```

```
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

5. Why Multiple Inheritance is Not Supported?

To avoid ambiguity problem.

```
class A {
    void show() {}
}

class B {
    void show() {}
}

// class C extends A, B -> ambiguity
```

Java supports multiple inheritance using interfaces.

6. Heap vs Stack Memory

Heap	Stack
Stores objects	Stores local variables
Shared memory	Thread-specific
Larger memory	Smaller memory

Example:

```
Employee emp = new Employee();
```

- emp reference stored in stack
- object stored in heap

7. Why Main Method is Static?

JVM can call it without creating object.

```
public static void main(String[] args)
```

8. Why String is Immutable?

Reasons:

1. Security
2. Thread safety
3. String pool optimization
4. HashMap key stability

Example:

```
String s = "Hello";  
s.concat("World");  
  
System.out.println(s);
```

Output:

```
Hello
```

9. String vs StringBuilder vs StringBuffer

String	StringBuilder	StringBuffer
Immutable	Mutable	Mutable
Slow	Fast	Thread-safe
Thread-safe	Not thread-safe	Thread-safe

Example:

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");
```

10. == vs equals()

==

Compares reference.

equals()

Compares content.

```
String s1 = new String("Java");  
String s2 = new String("Java");  
  
System.out.println(s1 == s2);  
System.out.println(s1.equals(s2));
```

Output:

```
false  
true
```

11. ArrayList vs LinkedList

ArrayList	LinkedList
Dynamic array	Doubly linked list
Fast random access	Fast insertion/deletion
Slow insertion in middle	Slow access

12. Internal Working of HashMap

Steps:

1. hashCode() called
2. Bucket calculated
3. Collision handled using LinkedList/Tree
4. Key compared using equals()

Example:

```
Map<Integer, String> map = new HashMap<>();  
map.put(1, "Java");
```

Java 8 uses Red-Black Tree when bucket size exceeds threshold.

13. HashMap vs Hashtable vs ConcurrentHashMap

HashMap	Hashtable	ConcurrentHashMap
Not thread-safe	Thread-safe	Thread-safe
Faster	Slower	Better concurrency
Allows null	No null	No null

14. HashSet Internal Working

HashSet internally uses HashMap.

```
Set<String> set = new HashSet<>();  
set.add("Java");
```

Duplicate prevention based on:

1. hashCode()
 2. equals()
-

15. Comparable vs Comparator

Comparable

Natural sorting.

```
class Employee implements Comparable<Employee> {
    int salary;

    public int compareTo(Employee e) {
        return this.salary - e.salary;
    }
}
```

Comparator

Custom sorting.

```
Comparator<Employee> comp = (a, b) -> a.salary - b.salary;
```

16. equals() and hashCode() Contract

If two objects are equal, hashCode must be same.

Example:

```
class Employee {
    int id;

    @Override
    public boolean equals(Object o) {
        Employee e = (Employee) o;
        return this.id == e.id;
    }

    @Override
    public int hashCode() {
        return id;
    }
}
```

```
}  
}
```

17. Checked vs Unchecked Exception

Checked	Unchecked
Checked at compile time	Checked at runtime
IOException	NullPointerException

18. throw vs throws

throw

Used to explicitly throw exception.

```
throw new RuntimeException();
```

throws

Declares exception.

```
void read() throws IOException {}
```

19. finally vs finalize

finally	finalize
Block	Method
Used for cleanup	Called by GC
Always executes	Deprecated

20. Process vs Thread

Process	Thread
Independent execution	Lightweight unit
Separate memory	Shared memory

21. Ways to Create Thread

Extending Thread

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running");  
    }  
}
```

Implementing Runnable

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Runnable thread");  
    }  
}
```

22. Runnable vs Callable

Runnable	Callable
No return value	Returns value
Cannot throw checked exception	Can throw checked exception

23. What is Synchronization?

Used to prevent race condition.

```
synchronized void increment() {  
    count++;  
}
```

24. What is Deadlock?

Two threads waiting for each other forever.

Example:

```
Thread 1 -> lock A -> waiting for B  
Thread 2 -> lock B -> waiting for A
```

25. wait() vs sleep()

wait()	sleep()
Releases lock	Does not release lock
Object class	Thread class

26. What is volatile?

Ensures visibility of variable changes across threads.

```
volatile boolean flag = true;
```

27. What is ExecutorService?

Manages thread pool.

```
ExecutorService service = Executors.newFixedThreadPool(5);
```

28. Functional Interface

Interface with single abstract method.

```
@FunctionalInterface
interface Test {
    void show();
}
```

29. Lambda Expression

```
Test t = () -> System.out.println("Hello");
```

30. Collection vs Stream

Collection	Stream
Stores data	Processes data
Reusable	Cannot reuse

31. map() vs flatMap()

map()

One-to-one mapping.

flatMap()

Flattens nested structure.

Example:

```
List<List<String>> list = List.of(
    List.of("A", "B"),
```

```
List.of("C", "D")
);

list.stream()
    .flatMap(Collection::stream)
    .forEach(System.out::println);
```

32. Why max() returns Optional?

To avoid NullPointerException.

```
OptionalInt max = list.stream()
    .mapToInt(x -> x)
    .max();
```

33. Optional.of vs Optional.ofNullable

of	ofNullable
Does not allow null	Allows null

34. JVM vs JRE vs JDK

JVM	JRE	JDK
Executes bytecode	Runtime environment	Development kit

35. JVM Memory Areas

1. Heap
 2. Stack
 3. Method Area
 4. Metaspace
 5. PC Register
-

36. What is Garbage Collection?

Automatic memory cleanup process.

Objects without reference become eligible for GC.

```
Employee e = new Employee();  
e = null;
```

37. Minor GC vs Major GC vs Full GC

Minor GC	Major GC	Full GC
Young generation	Old generation	Entire heap
Faster	Slower	Very slow

38. What is Memory Leak?

Unused objects still referenced.

Example:

```
List<Object> list = new ArrayList<>();  
  
while(true) {  
    list.add(new Object());  
}
```

39. How to Create Immutable Class?

```
final class Employee {  
    private final String name;  
  
    public Employee(String name) {  
        this.name = name;  
    }  
}
```

```
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Rules:

1. Class final
2. Variables private final
3. No setters

40. Why Immutable Objects are Thread-safe?

State cannot change after creation.

No synchronization required.

41. What is Serialization?

Converting object into byte stream.

```
class Employee implements Serializable {  
}
```

42. transient Keyword

Prevents variable from serialization.

```
transient String password;
```

43. serialVersionUID

Version control for serialized class.

```
private static final long serialVersionUID = 1L;
```

44. Reverse String

```
String str = "Java";

String reversed = new StringBuilder(str)
    .reverse()
    .toString();
```

45. Palindrome Check

```
String str = "madam";

boolean ans = str.equals(
    new StringBuilder(str).reverse().toString()
);
```

46. Find Duplicates using Stream

```
List<Integer> list = List.of(1,2,3,2,4,1);

Set<Integer> set = new HashSet<>();

list.stream()
    .filter(x -> !set.add(x))
    .forEach(System.out::println);
```

47. Find Second Highest Salary

```
employees.stream()
    .map(Employee::getSalary)
```

```
.distinct()  
.sorted(Comparator.reverseOrder())  
.skip(1)  
.findFirst();
```

48. Frequency Map using Stream

```
Map<String, Long> map = names.stream()  
    .collect(Collectors.groupingBy(  
        x -> x,  
        Collectors.counting()  
    ));
```

49. Why HashMap is Not Thread-safe?

Multiple threads can modify structure simultaneously causing inconsistent data.

Use ConcurrentHashMap in multithreaded environment.

50. Why ConcurrentHashMap is Faster?

Allows multiple threads to read/write simultaneously.

Java 8 uses:

1. CAS
2. Bucket-level locking

51. Why wait() must be inside synchronized?

Because thread must own monitor lock before calling wait().

52. Why Stream Cannot be Reused?

Once terminal operation executes, stream gets consumed.

```
Stream<Integer> stream = list.stream();
stream.count();
stream.count();
```

Second call throws exception.

53. What is Fail-fast Iterator?

Throws ConcurrentModificationException when collection modified during iteration.

```
List<Integer> list = new ArrayList<>();

for(Integer i : list) {
    list.add(5);
}
```

54. What is ThreadLocal?

Provides thread-specific variable.

```
ThreadLocal<Integer> local = new ThreadLocal<>();
```

55. Real Project Answers

Where did you use ConcurrentHashMap?

"We used ConcurrentHashMap in our backend application where multiple users updated shipment data simultaneously. It helped us avoid synchronization bottlenecks and improved concurrent performance."

56. Why did you use StringBuilder?

"We used StringBuilder while generating dynamic SQL queries and report content because String concatenation creates multiple immutable objects and impacts performance."

57. How did you optimize performance?

1. Used pagination
 2. Reduced DB calls
 3. Used caching
 4. Optimized streams
 5. Used batch processing
-

58. How did you handle exceptions globally?

Using Spring Boot:

```
@RestControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public String handle(Exception e) {
        return e.getMessage();
    }
}
```

59. Difference between synchronized and Lock

synchronized	Lock
Keyword	Interface
Automatic release	Manual release
Less flexible	More flexible

60. Why Java is Pass-by-value?

Java passes copy of variable.

For objects, reference copy is passed.

61. What Happens When Java Program Runs?

1. Java source compiled into bytecode
 2. ClassLoader loads class
 3. JVM verifies bytecode
 4. Memory allocated
 5. Execution starts from main method
-

62. What is ClassLoader?

Loads classes into JVM.

Types:

1. Bootstrap
 2. Extension
 3. Application
-

63. What is CAS?

Compare And Swap.

Used in concurrent programming without locking.

64. What is Stop-the-world Event?

Application threads pause during GC.

65. What is Rehashing?

When HashMap capacity exceeds threshold, bucket size increases and entries redistributed.

66. Why One Null Key Allowed in HashMap?

Null key always stored in first bucket.

67. Why ArrayList is Faster for Read?

Because it uses index-based array access.

Time Complexity:

```
get() -> O(1)
```

68. Why LinkedList Faster for Insert/Delete?

Only pointer updates required.

69. What is Race Condition?

Multiple threads modifying shared resource simultaneously.

70. What is Atomicity?

Operation either completes fully or not at all.

71. What is Method Reference?

Shortcut for lambda.

```
list.forEach(System.out::println);
```

72. What is Functional Programming?

Programming using functions and immutability.

73. Difference Between findFirst() and findAny()

findFirst	findAny
Returns first element	Returns any element
Mostly sequential	Optimized for parallel streams

74. anyMatch vs allMatch vs noneMatch

```
list.stream().anyMatch(x -> x > 5);  
list.stream().allMatch(x -> x > 5);  
list.stream().noneMatch(x -> x > 5);
```

75. Most Important Interview Tip

For every answer explain:

1. Definition
2. Internal working
3. Real-time usage
4. Advantages
5. One practical example

This makes you sound experienced instead of theoretical.